

Instituto Tecnológico de Colima

Ingeniería en Inteligencia Artificial

Cómputo en la Nube

Proyecto Nebula Arena — Unidad 3

Práctica 3.3 — Contenerizar y orquestar el servidor de Nebula Arena

Jarom Cortes Perez

Carlos Antonio Vogel Romero

11 de noviembre de 2026

Introducción

El servidor de partida es el corazón del juego: recibe la posición de los 10 jugadores y les reenvía el estado 20 veces por segundo. Para escalarlo en la nube hay que meterlo en un contenedor y dejar que un orquestador cree y borre servidores según haga falta. Para juegos, ese orquestador es Agones, que corre sobre Kubernetes y sabe cuándo un servidor tiene una partida en curso y no se debe tocar. En esta práctica empaquetamos el servidor, lo probamos en la laptop y lo desplegamos como flota de Agones en Amazon EKS.

Objetivo

Construir la imagen del servidor de partida, comprobar sus métricas, desplegarlo como flota de Agones, asignar un servidor a una partida y explicar qué pasa si se borra un servidor que tiene una partida en curso.

Lo que hicimos

1. El servidor de partida

Partimos del servidor mínimo del material (`server/server.py`, Python con asyncio y WebSocket) y conservamos su protocolo y los nombres de sus métricas, porque las gráficas de la práctica 3.4 dependen de ellos. Acepta hasta 10 jugadores (al 11.º lo rechaza con el código 4000, "partida llena") y responde los `ping` al instante para medir latencia. Le agregamos tres cosas:

- **Validación:** descarta JSON roto, valores fuera de la arena, `NaN` e ids inválidos sin tumbar la conexión.
- **Anti-suplantación:** cada conexión queda ligada a su jugador y no puede mover a otro.
- **Métricas extra:** tick rate efectivo, desconexiones por motivo y mensajes inválidos.

También le pusimos el ciclo de vida de Agones: avisa `/ready` al arrancar, manda `/health` cada 2 segundos y llama a `/shutdown` cuando termina la partida. Las 13 pruebas automáticas pasaron.

2. La imagen de Docker

Escribimos el `Dockerfile` sobre `python:3.11-slim`, con un usuario sin privilegios; la imagen pesa 206 MB. La construimos, la corrimos, conectamos 10 jugadores de prueba con `cliente_prueba.py` y leímos `/metrics`:

```
Evidencia práctica 3.3: contenedor del servidor en la laptop (Docker Desktop, Windows 11), 2026-11-11 19:53

> docker run -d --name nebula-evidencia -p 7654:7654 -p 9100:9100 nebula-server:1.2
9f13a86a42f4a3e976f819d0dd1dc8c9f78fc8305ad0bf1406c4d4573f209d46

> docker ps --filter name=nebula-evidencia
CONTAINER ID   IMAGE               STATUS         PORTS
9f13a86a42f4   nebula-server:1.2   Up 4 seconds   0.0.0.0:7654->7654/tcp, [::]:7654->7654/tcp, 0.0.0.0:9100->9
100/tcp, [::]:9100->9100/tcp

> docker logs nebula-evidencia
2026-11-12 01:53:08,564 INFO websockets.server: server listening on 0.0.0.0:7654
2026-11-12 01:53:08,564 INFO nebula: escuchando en :7654 (metricas :9100)
2026-11-12 01:53:08,573 INFO nebula: SDK de Agones no disponible todavía; se reintenta /ready cada 1 s

> python server/cliente_prueba.py --url ws://localhost:7654 --jugadores 10 --segundos 10
10 jugadores, 10 s, 90 pings
RTT p50=5.33 ms  p95=48.68 ms  p99=51.01 ms

> curl -s localhost:9100/metrics | grep ^nebula
nebula_players_connected 0.0
nebula_tick_seconds_bucket{le="0.001"} 127.0
nebula_tick_seconds_bucket{le="0.0025"} 313.0
nebula_tick_seconds_bucket{le="0.005"} 319.0
nebula_tick_seconds_bucket{le="0.01"} 319.0
nebula_tick_seconds_bucket{le="0.025"} 319.0
nebula_tick_seconds_bucket{le="0.05"} 319.0
nebula_tick_seconds_bucket{le="0.1"} 319.0
nebula_tick_seconds_bucket{le="+Inf"} 319.0
```

Figura 1. *docker run del servidor en la laptop, 10 jugadores de prueba y las métricas en /metrics: tick rate 19.66 de 20, 10 desconexiones normales y 0 mensajes inválidos.*

El aviso "SDK de Agones no disponible todavía" es normal en la laptop: ahí no existe Agones. En el clúster ese mismo reintento resolvió un problema real (ver la práctica 3.4).

En la laptop el contenedor dio una latencia p95 de 49 a 75 ms, contra 4 ms del mismo servidor sin Docker. La diferencia la agrega el reenvío de puertos de Docker Desktop en Windows, no el servidor; en la nube esa capa no existe.

3. Ensayo en un clúster local

Antes de pagar nube montamos un clúster local con kind y Agones 1.62 para ensayar la flota. Los tres servidores llegaron a `Ready` en 20 segundos, y al crear una `GameServerAllocation` uno pasó a `Allocated`:

```
## 1. Antes de la asignación
$ kubectl get gameservers
NAME                       STATE   ADDRESS      PORT   NODE                   AGE
nebula-fleet-v9js2-4t5ff   Ready   172.18.0.2   7052   nebula-control-plane   33s
nebula-fleet-v9js2-dcxrp   Ready   172.18.0.2   7724   nebula-control-plane   33s
nebula-fleet-v9js2-gdgj6   Ready   172.18.0.2   7722   nebula-control-plane   33s

## 2. Asignación
$ kubectl create -f k8s/allocation.yaml
gameServerName: nebula-fleet-v9js2-4t5ff
$ kubectl get gameservers
NAME                       STATE       ADDRESS      PORT   NODE                   AGE
nebula-fleet-v9js2-4t5ff   Allocated   172.18.0.2   7052   nebula-control-plane   34s
nebula-fleet-v9js2-dcxrp   Ready       172.18.0.2   7724   nebula-control-plane   34s
nebula-fleet-v9js2-gdgj6   Ready       172.18.0.2   7722   nebula-control-plane   34s
```

Figura 2. *Ensayo local en kind: kubectl get gameservers antes y después de crear la GameServerAllocation (un servidor pasa de Ready a Allocated).*

4. La infraestructura en AWS

La cuenta de Google Cloud nos pidió un prepago, así que hicimos la práctica en AWS, en la región us-east-2 (la única que permite la cuenta del plan gratuito). Con Terraform creamos:

- Dos repositorios de ECR para las imágenes.
- Una red con dos subredes públicas y sin NAT Gateway, lo que ahorra unos USD 33 al mes, con los puertos 7000–8000 abiertos para las partidas.
- Un clúster EKS 1.36 con nodos `m7i-flex.large` (2 vCPU y 8 GB, permitidos por el plan gratuito), de 2 a 6 nodos.
- Agones 1.62 instalado con Helm.

Subimos la imagen a ECR y aplicamos la flota con 3 réplicas. Los **3 servidores estuvieron en `Ready` a los 8 segundos**:

```
> kubectl get gameservers (imagen desplegada por el pipeline)
NOMBRE                     ESTADO   IMAGEN
nebula-fleet-h9d2j-6hxnk   Ready    ********1139.dkr.ecr.us-east-2.amazonaws.com/nebula-server:c295bf8dd34140f
bc7461c63943eba8b063aacf2
nebula-fleet-h9d2j-dn7px   Ready    ********1139.dkr.ecr.us-east-2.amazonaws.com/nebula-server:c295bf8dd34140f
bc7461c63943eba8b063aacf2
nebula-fleet-h9d2j-ntslt   Ready    ********1139.dkr.ecr.us-east-2.amazonaws.com/nebula-server:c295bf8dd34140f
bc7461c63943eba8b063aacf2
```

Figura 3. *kubectl get gameservers ANTES de la asignación, en EKS: la flota con sus 3 servidores en Ready.*

5. Asignación y partida real por internet

Para asignar servidores escribimos un matchmaker: un servicio que recibe `POST /match`, le pide a Agones un servidor libre y le devuelve al jugador la dirección y el puerto. Desde la PC, en Colima, le pedimos una partida y conectamos 10 jugadores durante 15 segundos al servidor que nos dio:

```
Partida real en EKS a traves del matchmaker, 2026-11-09 14:24

> POST http://a6cd6956a66f14954a5b0517b09e7786-2138136351.us-east-2.elb.amazonaws.com/match {"jugador":"carlo
s"}
{
    "address":  "ec2-3-139-56-74.us-east-2.compute.amazonaws.com",
    "gameServerName":  "nebula-fleet-nrpzj-2nt49",
    "port":  7981
}

> cliente_prueba.py --jugadores 10 --segundos 15 (desde la PC del alumno, por internet)
10 jugadores, 15 s, 130 pings
RTT p50=106.30 ms  p95=133.42 ms  p99=147.01 ms

> kubectl get gameservers
NAME                       STATE       ADDRESS                                          PORT   NODE
AGE
nebula-fleet-nrpzj-2nt49   Shutdown    ec2-3-139-56-74.us-east-2.compute.amazonaws.com   7981   ip-10-0-31-58.
us-east-2.compute.internal   103s
nebula-fleet-nrpzj-66hbb   Ready       ec2-3-139-56-74.us-east-2.compute.amazonaws.com   7610   ip-10-0-31-58.
us-east-2.compute.internal   103s
nebula-fleet-nrpzj-68727   Ready       ec2-3-139-56-74.us-east-2.compute.amazonaws.com   7041   ip-10-0-31-58.
us-east-2.compute.internal   103s
nebula-fleet-nrpzj-rq88t   Scheduled   ec2-3-139-56-74.us-east-2.compute.amazonaws.com   7145   ip-10-0-31-58.
us-east-2.compute.internal   0s
```

Figura 4. *Asignación por el matchmaker y partida real por internet. Al final, kubectl get gameservers DESPUÉS: el servidor asignado terminó su partida (Shutdown) y la flota ya está creando su reemplazo (Scheduled).*

La latencia fue de **106 ms (p50)**, 133 ms (p95) y 147 ms (p99), que es básicamente la distancia de Colima a Ohio. Cuando los jugadores salieron, el servidor llamó a `/shutdown`, pasó a `Shutdown`, y la flota creó uno nuevo para volver a tener 3 libres. Así comprobamos el ciclo completo `Ready → Allocated → Shutdown`.

Pregunta: ¿qué pasa si borras un pod en estado Allocated?

Lo probamos en el clúster local. Un servidor `Allocated` tiene una partida en curso, y su estado (posiciones y puntos) vive solo en la memoria de ese proceso. Al borrar el pod a mano:

1. Kubernetes manda `SIGTERM` a los contenedores, se cortan las conexiones de los jugadores y **la partida se pierde**, porque no hay otra copia de ese estado.
2. Mientras el pod se apaga, el `GameServer` sigue diciendo `Allocated`, hasta que Agones detecta que dejó de responder (`Unhealthy`) y lo borra.
3. La flota ve que le falta una réplica y crea un servidor **nuevo** en `Ready`. No es una recuperación: el servidor nuevo está vacío y la partida no regresa.

```
## 7. Experimento del entregable: borrar a mano un pod Allocated
$ kubectl create -f k8s/allocation.yaml  -> nebula-fleet-v9js2-cxphd
NAME                        STATE       ADDRESS      PORT   NODE                   AGE
nebula-fleet-v9js2-cxphd   Allocated   172.18.0.2   7007   nebula-control-plane   33s
nebula-fleet-v9js2-dcxrp   Ready       172.18.0.2   7724   nebula-control-plane   89s
nebula-fleet-v9js2-gdgj6   Ready       172.18.0.2   7722   nebula-control-plane   89s
$ kubectl delete pod nebula-fleet-v9js2-cxphd --wait=false
pod "nebula-fleet-v9js2-cxphd" deleted from default namespace
$ kubectl get gameservers   (3 s después)
NAME                        STATE       ADDRESS      PORT   NODE                   AGE
nebula-fleet-v9js2-cxphd   Allocated   172.18.0.2   7007   nebula-control-plane   37s
nebula-fleet-v9js2-dcxrp   Ready       172.18.0.2   7724   nebula-control-plane   93s
nebula-fleet-v9js2-gdgj6   Ready       172.18.0.2   7722   nebula-control-plane   93s
$ kubectl get events --field-selector involvedObject.name=nebula-fleet-v9js2-cxphd
[…]
## 8. ~60 s después del borrado (el pod agotó los 30 s de gracia: python como PID 1 ignoraba SIGTERM)
$ kubectl get gameservers
NAME                        STATE   ADDRESS      PORT   NODE                   AGE
nebula-fleet-v9js2-dcxrp   Ready   172.18.0.2   7724   nebula-control-plane   2m31s
nebula-fleet-v9js2-gdgj6   Ready   172.18.0.2   7722   nebula-control-plane   2m31s
nebula-fleet-v9js2-tzkfc   Ready   172.18.0.2   7977   nebula-control-plane   30s
$ kubectl get events --field-selector involvedObject.name=nebula-fleet-v9js2-cxphd
61s         Normal    Killing          pod/nebula-fleet-v9js2-cxphd          Stopping container nebula
31s         Warning   Unhealthy        pod/nebula-fleet-v9js2-cxphd          Liveness probe failed: Get "htt
p://10.244.0.17:8080/healthz": dial tcp 10.244.0.17:8080: connect: connection refused
30s         Warning   Unhealthy        gameserver/nebula-fleet-v9js2-cxphd   Issue with Gameserver pod
```

Figura 5. *Experimento: borrar a mano un pod Allocated. Los eventos muestran Killing, Unhealthy y Deletion started; al final la flota tiene un servidor nuevo (tzkfc) en Ready.*

En el experimento encontramos un error nuestro: el pod tardó **30 segundos** en morir. En el contenedor, Python es el proceso 1 y, sin un manejador, ignora `SIGTERM`, así que Kubernetes esperó todo el periodo de gracia. En la versión 1.1 agregamos el manejo de `SIGTERM` y el mismo borrado tardó **2 segundos**.

Por eso nunca hay que borrar a mano un servidor `Allocated`. Agones protege esos servidores del escalado y de las actualizaciones, y espera a que el propio servidor llame a `/shutdown` al terminar la partida.

Conclusión

El servidor quedó empaquetado en una imagen pequeña y segura, y Agones lo administra como flota: arranca servidores en segundos, asigna uno por partida y repone los que terminan. Probar primero en un clúster local nos ahorró dinero y nos dejó encontrar el problema de `SIGTERM` antes de llegar a la nube. La partida real por internet confirmó que todo funciona fuera de la laptop, y que la latencia base desde Colima es de unos 106 ms por la distancia a la región.